

セキュリティ実践演習第一 第2回 マルウェア解析

理工学部理工学科

知能情報システムプログラム

DX人材育成基盤プログラム

池部実 minoru@oita-u.ac.jp

マルウェア解析についての演習

- マルウェア解析(1) 表層解析
- マルウェア解析(2) 逆アセンブリ
- マルウェア解析(3) 動的解析
- マルウェア解析(4) コードからの特徴抽出
- マルウェア解析(5) 機械学習・深層学習によるマルウェア分類
- マルウェア対策・攻撃検知

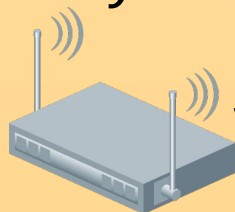
演習内容

- 機械学習などを用いてマルウェアを分類
 - 特徴量抽出
- 演習を踏まえて、マルウェア対策・攻撃検知のための方法を検討する

演習環境(1)

example.jp.
10.0.0.0/24

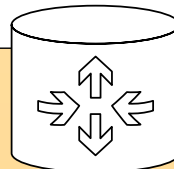
SSID: SecurityLab
(hidden)



Ubuntu
DHCP
10.0.0.0/24



ルータ



binary.example.jp.
10.0.0.160/24
(Ubuntu Server)

closed network!!
can't connect the Internet

準備(1)

- 演習端末(DELL, Ubuntu)
 - login id: security
 - password: *****(Moodle上資料では非公開)
 - 端末に記載
 - 無線LAN: SecurityLab 接続(済)
- 演習用NWは, B-Coreで演習端末からのみ接続可
- 時間外に演習用サーバに接続する場合, 踏み台サーバを利用する(後述)

準備(2)

- サーバには, Ubuntu(ノートPC)のVSCodeから `ssh(extension)` で接続
- 接続先サーバ: `binary.example.jp`.
- サーバには, `security` ユーザでログイン
 - パスワードはUbuntu(ノートPC)と同じ
- `binary.example.jp`上で実行したウィンドウを手元のUbuntu(ノート)で表示したい場合,
`ssh -X binary.example.jp`
`ssh -Y binary.example.jp`
としてSSH接続する

~/ .ssh/config

- binary.example.jpで実行したmatplotlibのウィンドウをUbuntu(ノートPC)上に表示
- ~/ .ssh/configに以下のように記述

```
Host binary.example.jp
  HostName binary.example.jp
  User security
  ForwardX11 yes
  ForwardX11Trusted yes
```

(注意事項)Ubuntu上でのpip

- UbuntuなどのLinuxや, macOSでは共通環境へのpipインストールは推奨されていない
- ユーザ毎にvirtualenvの利用を推奨

```
python3 -m pip install matplotlib
error: externally-managed-environment
```

```
× This environment is externally managed
↳ To install Python packages system-wide, try apt install
python3-xyz, where xyz is the package you are trying to
install.
```

If you wish to install a non-Debian-packaged Python package, create a virtual environment using `python3 -m venv path/to/venv`. Then use `path/to/venv/bin/python` and `path/to/venv/bin/pip`. Make sure you have `python3-full` installed.

If you wish to install a non-Debian packaged Python application, it may be easiest to use `pipx install xyz`, which will manage a virtual environment for you. Make sure you have `pipx` installed.

See `/usr/share/doc/python3.12/README.venv` for more information.

note: If you believe this is a mistake, please contact your Python installation or OS distribution provider. You can override this, at the risk of breaking your Python installation or OS, by passing `--break-system-packages`.

hint: See PEP 668 for the detailed specification.

binary.example.jp

- sshでログインすると, venv環境を自動で読み込むように設定
 - (入力の必要なし) `source ~/.venv/bin/activate`
 - プロンプトの先頭に(.venv)が付いていればOK
- pipでインストールの際
 - `$ python3 -m pip MODULENAME`
- 実行に必要なモジュールはpipでインストール済み

演習内容

- マルウェアの種類推定を例とした,
機械学習を用いたバイナリ解析

バイナリ解析レポート

- 今回の演習では、`binary.example.jp`上でのターミナルでの実行結果をすべてテキストファイルに記載しておくこと

VSCodeでbinary.example.jp に接続

- ~/m12ディレクトリをオープン
- ターミナルを操作して、PythonのインタラクティブシェルでPythonのプログラムを実行していく
- VSCodeでPythonプログラムを作成
 - (踏み台経由はvi)

マルウェア(Malware)

- 悪意のあるソフトウェア(Malicious Software)の略称
- コンピュータシステムに害を与えることを目的として作成されたプログラムの総称
 - 個人情報や機密データの窃取
 - システムの不正操作
 - ランサムウェアによる金銭の要求

マルウェアの分類方法

- 感染・増殖方法に焦点をあてた分類
- 機能・目的に焦点をあてた分類

感染・増殖方法に焦点をあてた分類

- ウイルス

- 他のプログラムの一部を改ざんして入り込み、増殖するマルウェア

- ワーム

- 自身をコピーし、ネットワーク等を介して他のコンピュータに拡散するマルウェア

- トロイの木馬

- 無害なプログラムを装い端末に侵入して、裏で悪意のある動作するマルウェア

機能・目的に焦点をあてた分類

- RAT(Remote Access Trojan)
 - 攻撃者がマウスやキーボード操作、ファイルやネットワークのリソースへのアクセスを含めて遠隔操作を可能にするマルウェア
- インフォスティーラー(Infostealer)
 - Information Stealer, 感染したデバイスから情報を窃取するマルウェア
- ランサムウェア
 - 感染したコンピュータのファイルなどを暗号化して正常に利用できなくするマルウェア
- ワイパー
 - 感染したファイルやディスクのデータを破壊して、正常に利用できなくするマルウェア
- ダウンローダ
 - 他のマルウェアをダウンロードするマルウェア
- ドロップ
 - 内部に持っているマルウェアを投下するマルウェア
- (注)
 - マルウェアによっては複数の機能が実装されているので1つのマルウェアが上記の機能・目的を複数もつ場合もある

マルウェア解析

- SOCの立場
 - 不審なファイルがマルウェアか否かを特定するために解析
 - 攻撃を検知できるように検知ルール作成のために解析
- セキュリティインシデント発生時
 - マルウェアを解析して、その目的や機能を特定できれば攻撃者の特定や効果的な防御策の考案につながる可能性
- マルウェア解析には膨大な時間と専門知識が必要
⇒ 機械学習の活用

機械学習

- コンピュータが大量のデータから自動的にパターンや知識を学習し，新しいデータに対して予測や分類するための技術
 - データから自動的に予測や分類のためのルールを発見すること
- 機械学習の種類
 - 教師あり学習
 - 教師なし学習
 - 強化学習

教師あり学習

- 入力データとそれに対応する正解ラベルのセット (学習データ) を用いてモデルを学習
- 学習
 - データを入力したときの出力となる正解値との誤差を計算し、その誤差を小さくするようにパラメータを更新する過程を繰り返すこと
- 学習済みモデル
 - 学習が終わったモデル
 - 識別や分類を目的としたものを分類器と呼ぶ
- 推論
 - 新しい入力データに対して予測や分類が可能となる過程
- 分類器の性能
 - 学習に使用しなかったテストデータを用いて評価

教師あり学習のステップ

- データの収集
- 前処理
- モデルの選択
- 学習
- 評価

深層学習(ディープラーニング)

- ニューラルネットワーク
 - 人間の脳の神経回路を模した機械学習モデル
 - 多層の構造を取り，層の数が多い場合を深層学習と呼んだりする
- 入力層
 - 入力データを受けとる層，画像認識の場合には各ピクセルの値などを受けとる
- 隠れ層 or 中間層
 - 入力層と出力層の間にある層で，データの特徴を抽出
 - 複数の中間層を重ねることで，モデルの表現力が増し，より複雑な特徴を学習できると言われている
- 出力層
 - 最終的な結果を出力する層，分類問題では各クラスの確率を出力することが一般的

マルウェア解析と機械学習

- マルウェア解析
 - 静的解析
 - プログラムを動作させずにマルウェアの機能や目的を調査する手法で，デコンパイルしてアセンブリ命令を分析
 - 表層解析はファイルのメタデータなどの表面的な情報からマルウェアの特徴を特定する手法
 - 動的解析
 - マルウェアを実際に実行して，そのきょどうを観測してマルウェアの機能や目的を洞察
- 今回は，前回につづいて静的解析

マルウェア解析用 データセットにおけるラベル

- 悪性ファイル・良性ファイル(マルウェア検知)
- マルウェアカテゴリ(ランサムウェア, ドロップなど)
- マルウェアファミリ(Emotet, Trickbotなど)
- 使用されているパッカー
- 使用されているコンパイラ
- など

機械学習によるマルウェア検知

- マルウェア分類のための特徴量
 - ファイルのメタデータ, バイナリデータのバイト列, アセンブリコードなど様々な特徴量が用いられている
 - 使用する特徴量によりマルウェア分類の方法は異なる
- メタデータを用いる手法
 - ヘッダ情報から特徴量を比較的に取得できる
 - 一方, メタデータの改ざんが容易

機械学習によるマルウェア検知

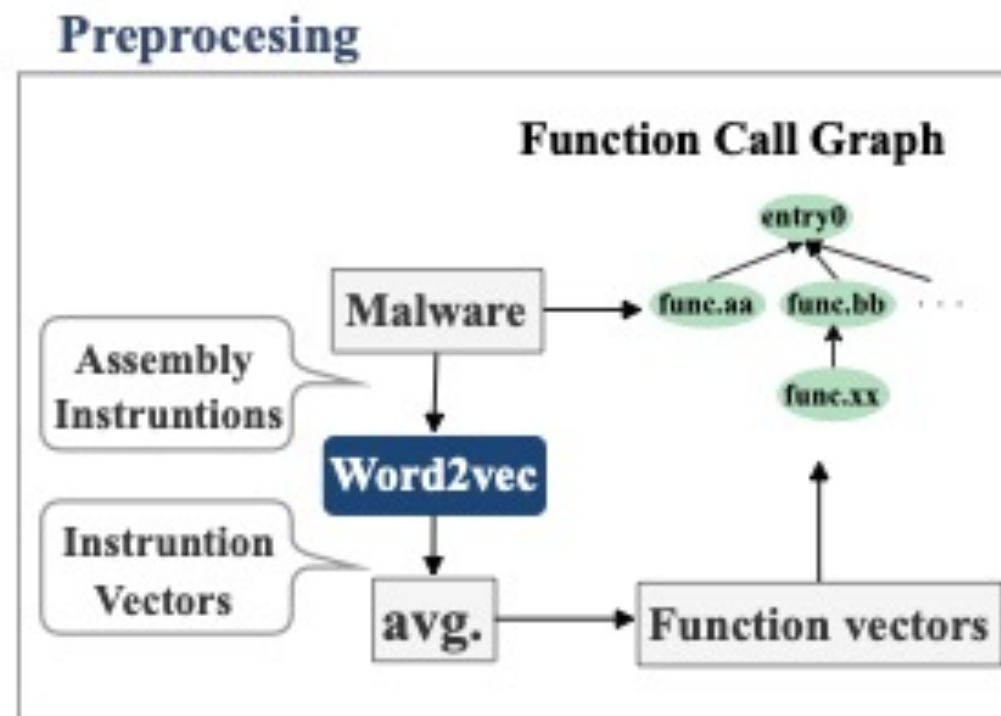
- マルウェアを画像化する手法
 - バイナリデータを画像に変換し，画像分類する方法
 - CNNが広く利用されており，既存の画像分類技術を適用できるメリット
- バイト列を用いる手法
 - バイト列を特徴量とする方法
 - 実行ファイルのバイナリコードをバイト単位で分割し，そのシーケンスを機械学習モデルの入力として利用

機械学習によるマルウェア検知

- アセンブリ命令を用いる手法
 - アセンブリ命令を特徴量として利用する方法
 - マルウェアがどのような処理をしているか, どのような制御フローを持っているのかなど, マルウェアの挙動を特徴づける手がかりが隠されている
 - 逆アセンブルして, 特徴量として扱う
 - FCGAT
 - 関数間の呼び出し関係をコールグラフと関数の特徴ベクトルを使用してマルウェア分類
 - Minami Someya (Institute of Information Security), Yuhei Otsubo (National Police Academy), Akira Otsuka (Institute of Information Security)
FCGAT: Interpretable Malware Classification Method using Function Call Graph and Attention Mechanism
 - <https://www.ndss-symposium.org/ndss-paper/auto-draft-396/>

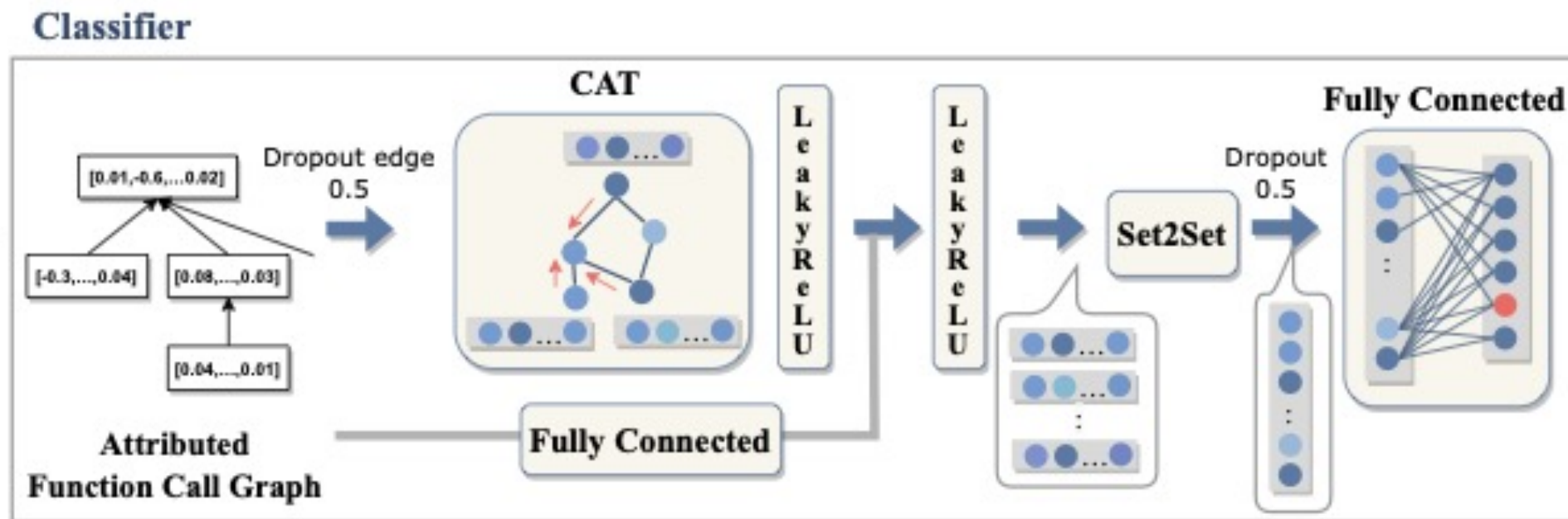
FCGAT(1)

- 関数同士の呼び出し関係を表現したコールグラフを作成し, word2vecで生成した各関数(ノード)の特徴ベクトルをコールグラフのノードの特徴量として割り当て



FCGAT(2)

- グラフニューラルネットワーク(GNN)でコールグラフの特徴を学習し，マルウェアの種類を分類



特徴量の作成

- 関数の特徴量をword2vecを利用して作成
- word2vec
 - NLP分野で単語をベクトル化するための手法
 - 単語を高次元のベクトルに埋め込むことで、単語間の類似度や関係性を数値的に捉えることができる
- word2vecでアセンブリ命令をベクトル化
 - 似た意味の命令を類似したベクトルで表現

CBOW(word2vecのモデル)

- CBOW(Continuous Bag of Words)モデル
 - ある単語の周囲のコンテキスト(単語)から, 中央の単語(ターゲット)を予測するように学習
- 例)
 - 明日は雨が降るかもしれません。
 - 単語「雨」をターゲットとすると, その周辺の単語列「明日は」と「が降るかもしれません」がコンテキスト
 - CBOWではこのコンテキストからターゲット「雨」を予想するように学習

CBOW(word2vecのモデル)

- アセンブリ命令を単語に，関数を文に対応させることが可能
- アセンブリ命令を関数のコンテキストでベクトル化し，似た意味や役割をもつ命令を類似したベクトルで表現可能
- 例) jmpとjzのようなジャンプ命令は近いベクトルで表現されることを期待

word2vec学習のためのデータ 準備

- word2vecの学習には大量のコーパス(文章のまとまり)が必要
- 今回のケースでは、実行ファイルから多くのアセンブリ命令列を抽出して学習コーパスを作成する必要
- ~/ml/bins にある実行ファイルからアセンブリ命令のコーパスを構築
- radare2(<https://rada.re/>)で関数ごとにアセンブリ命令を抽出

正規化

- word2vecでは学習データに含まれていない単語 (OOV: Out of Vocabulary)はベクトル化できない
- 例)
 - cmpやpushなどのオペコードは多くのサンプルに含まれているが、メモリアドレスやオフセットなどの値はサンプルごとに異なる場合が多い
- 固有値が残っていると、OOVとなって適切にベクトル化できない可能性
- メモリアドレスや特定の数値などを定数に置き換える正規化をすることで共通のパターンを見つけやすくする

正規化

- 今回の正規化例
 - 数値(16進数)をNに置き換える
 - オペコードとオペランドを_で繋ぐ
- 実装
 - `normalize_instruction()`関数
 - reモジュールを使用

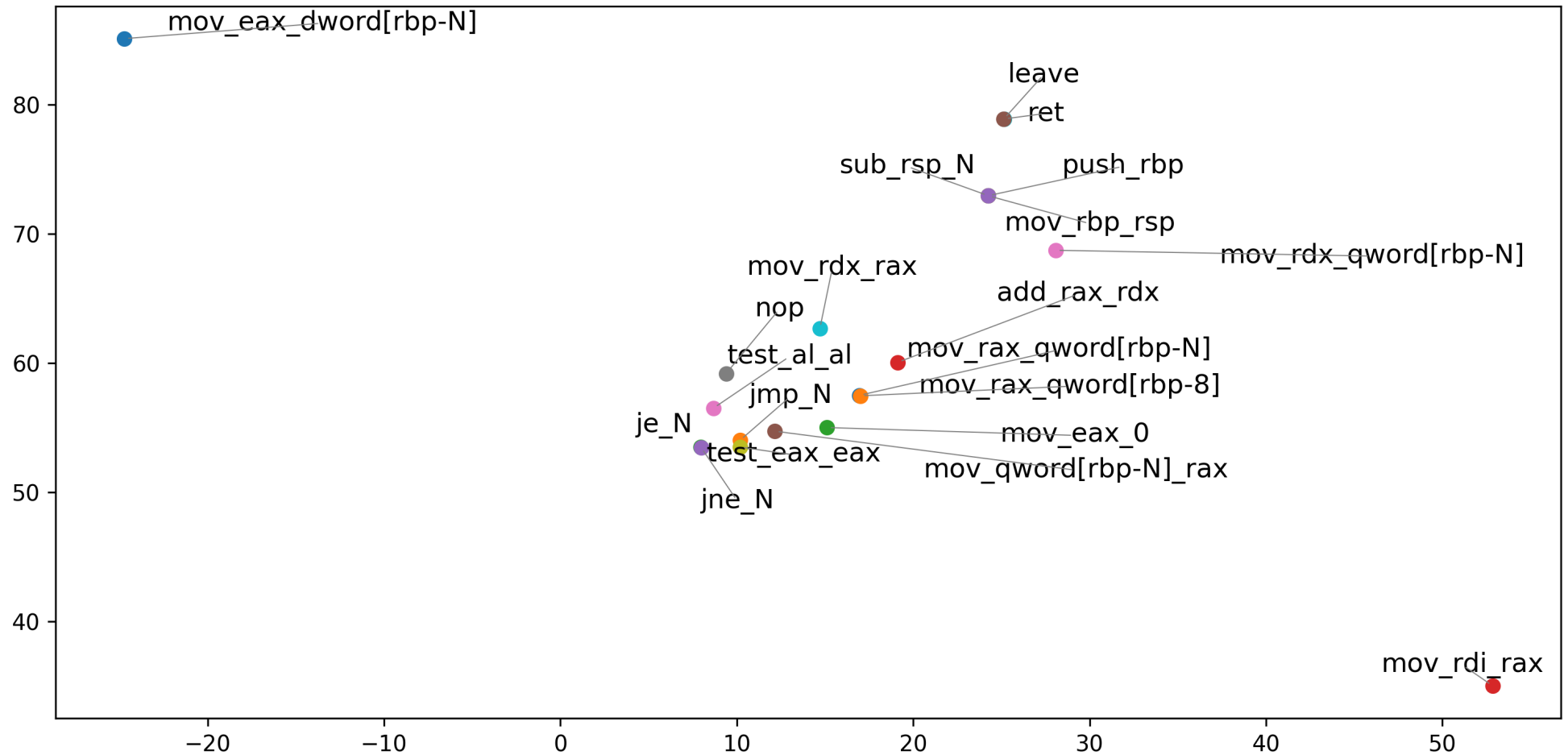
word2vecの学習

- 学習には, `gensim`ライブラリを使用
 - `gensim`: NLPのトピックモデリングや文書類似度の計算などに広く使用されるライブラリ
- `gensim.models`の`Word2Vec`クラスを使用
 - モデル: `assembly_word2vec.model` (ファイル保存)

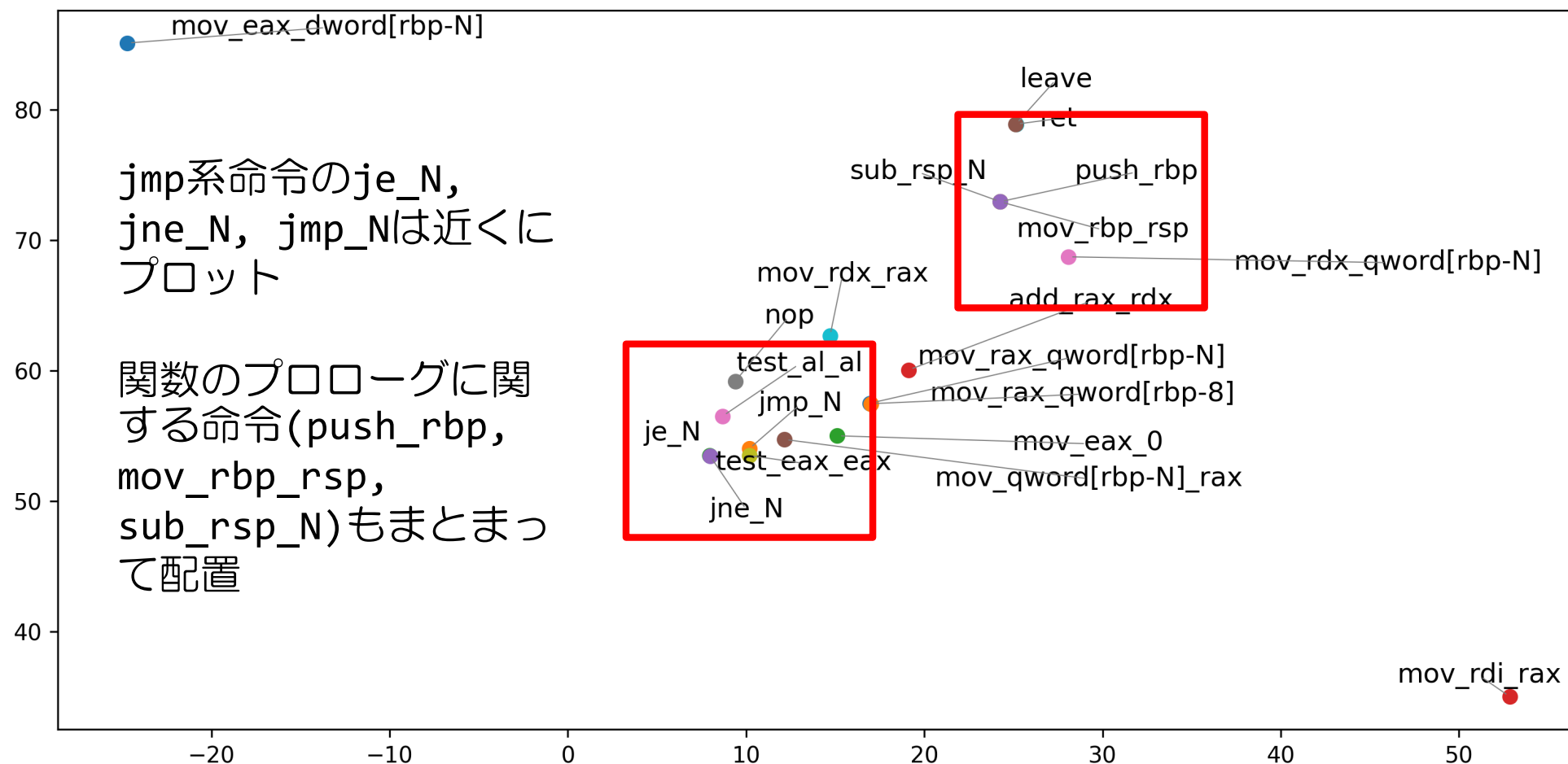
命令ベクトルの可視化

- 作成した命令ベクトル: 100次元
- t-SNE
 - 高次元のデータを低次元に変換して視覚化
 - データのクラスタリングや類似度を視覚的に確認可能
- t-SNEを用いてアセンブリ命令の100次元ベクトルを2次元にプロット
 - t-SNEモデルを学習し, ベクトルを2次元に圧縮
 - 結果を可視化

命令ベクトルの可視化



命令ベクトルの可視化



プログラムの実行

```
$ cd ~/ml
```

```
$ python3 malwareclassification.py
```

グラフニューラルネットワークを使用したマルウェア分類

- FCGAT
 - GNN(グラフニューラルネットワーク)を使用している点
 - グラフ構造のデータを処理するために設計されたニューラルネットワークの一種
- GNN
 - ノードとエッジからなるグラフデータを処理可能
 - SNS, 化合物, 通信ネットワークなど様々な応用分野で優れた性能を発揮
- 実行ファイル
 - グラフ構造で表現可能
 - 基本ブロック(分岐などがない一連の命令列)をノードとした制御フローグラフや関数をノードとしたコールグラフ

GNN

- 各ノードの特徴ベクトルを，隣接ノードの特徴ベクトルと組み合わせながら反復的に更新
- この過程を通じて，グラフの構造を捉えた特徴量を学習可能
- 例)
 - マルウェアの関数が他のマルウェアの関数と似たような呼び出しパターンを持っていれば，GNNはそれらの関数が似た特徴を有していると判断する可能性が高い
- GNNを用いることで関数間の関係性を考慮した分類が可能

fcgat.py

- データセットに含まれるマルウェアの種類
 - backdoor
 - downloader
 - dropper
 - ransomware
 - torojan
 - worm

学習と評価

- データセットを8:2で訓練データとテストデータに分割
 - 訓練ではデータをシャッフルして、モデルが毎エポックで異なる順序でデータを見ることができ、過学習を防ぐ
 - 本来は、交差検証が推奨されるが時間がかかるので今回は単純な分割

学習の実行

```
Epoch: 001, Train Loss: 1.6407, Train Acc: 0.6050, Test Acc: 0.5657
Epoch: 002, Train Loss: 1.2438, Train Acc: 0.6948, Test Acc: 0.6538
Epoch: 003, Train Loss: 0.9517, Train Acc: 0.7439, Test Acc: 0.7141
Epoch: 004, Train Loss: 0.8069, Train Acc: 0.7818, Test Acc: 0.7543
Epoch: 005, Train Loss: 0.7434, Train Acc: 0.7903, Test Acc: 0.7573
...
Epoch: 099, Train Loss: 0.1849, Train Acc: 0.9416, Test Acc: 0.9057
Epoch: 100, Train Loss: 0.1875, Train Acc: 0.9427, Test Acc: 0.9042
Training Time: 568.2830510139465
```

各エポックにおける訓練損失, 訓練精度, テスト精度が表示

エポックが進むにつれ, 訓練損失が減少し, 精度が向上していれば, 正常に学習が進んでいる

100エポック学習で, テスト精度が92%程度になる(はず)

プログラムの実行

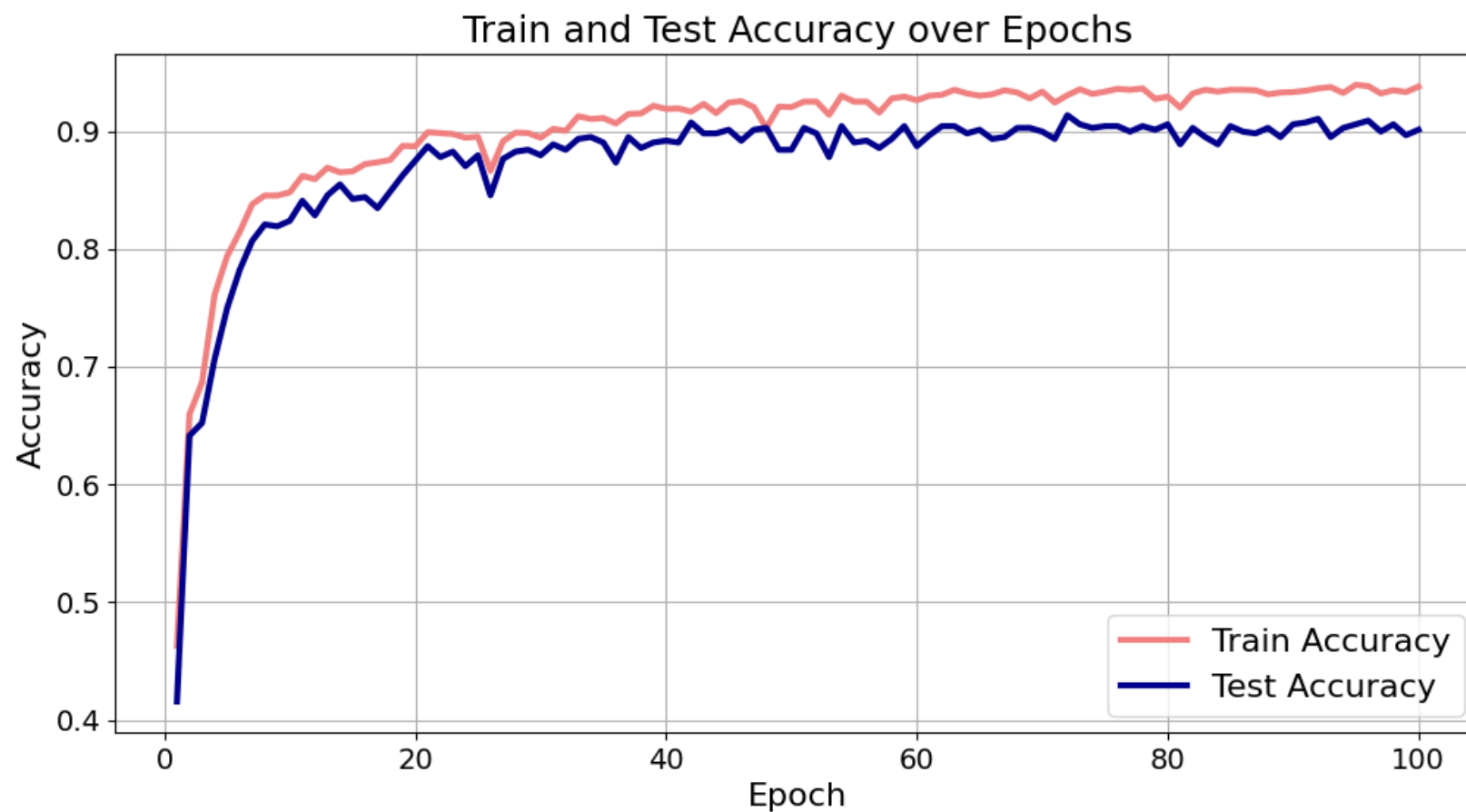
```
$ cd ~/ml
```

```
$ python3 fcgat.py
```

// それなりの時間がかかります

- PyTorchでは, GPUを利用
 - GPU : NVIDIA GeForce RTX 2070
 - 2018年ごろのグラフィックボード

學習曲線 (accuracy.png)



混合行列(confusionmatrix.png)

